ASSESSMENT – 10 THREE ADDRESS CODE GENERATOR

Compiler Design Lab | Academic Year 2026–27

Submission due date: 21 October 2026

Project overview

This project accepts arithmetic expressions containing identifiers, numeric constants, parentheses, and the operators +, -, * and /. It generates Three Address Code (TAC), Quadruples, Triples, and an Indirect Triple pointer sequence. It also reports basic syntax errors.

Team members

Priyanshu — 23BCE0221
Aniket Singh — 24BCE2100
Sahithi.P — 24BCE0170

1. Source Code

File name: **tac_generator.cpp** | Language: C++17

```
#include <iostream>
#include <string>
#include <vector>
#include <cctype>
#include <iomanip>
using namespace std;

struct Instruction {
    string op, arg1, arg2, result;
};

class TACGenerator {
private:
    string input;
    size_t pos;
    vector<Instruction> code;
    int tempCount;

    void skipSpaces() {
        while (pos < input.size() &&
               isspace((unsigned char)input[pos]))
            pos++;
    }

    bool match(char c) {
        skipSpaces();
        if (pos < input.size() && input[pos] == c) {
            pos++;
            return true;
        }
        return false;
    }

    string newTemp() {
        return "t" + to_string(++tempCount);
    }

    string parseFactor() {
        skipSpaces();

        if (match('(')) {
            string value = parseExpression();

            if (!match(')'))
                throw string("Missing closing parenthesis.");

            return value;
        }

        if (pos < input.size() &&
            (isalpha((unsigned char)input[pos]) ||
             input[pos] == '_')) {

            size_t start = pos++;

            while (pos < input.size() &&
                   (isalnum((unsigned char)input[pos]) ||
```

```
                input[pos] == '_'))
            pos++;

        return input.substr(start, pos - start);
    }

    if (pos < input.size() &&
        isdigit((unsigned char)input[pos])) {

        size_t start = pos++;

        while (pos < input.size() &&
               isdigit((unsigned char)input[pos]))
            pos++;

        if (pos < input.size() && input[pos] == '.') {
            pos++;

            while (pos < input.size() &&
                   isdigit((unsigned char)input[pos]))
                pos++;
        }

        return input.substr(start, pos - start);
    }

    throw string("Invalid arithmetic expression.");
}

string parseTerm() {
    string left = parseFactor();

    while (true) {
        skipSpaces();

        if (pos < input.size() &&
            (input[pos] == '*' || input[pos] == '/')) {

            char op = input[pos++];
            string right = parseFactor();
            string temp = newTemp();

            code.push_back({
                string(1, op), left, right, temp
            });

            left = temp;
        } else {
            break;
        }
    }

    return left;
}

string parseExpression() {
    string left = parseTerm();

    while (true) {
        skipSpaces();
```

```
            if (pos < input.size() &&
                (input[pos] == '+' || input[pos] == '-')) {

                char op = input[pos++];
                string right = parseTerm();
                string temp = newTemp();

                code.push_back({
                    string(1, op), left, right, temp
                });

                left = temp;
            } else {
                break;
            }
        }

        return left;
    }

public:
    explicit TACGenerator(const string& expr)
        : input(expr), pos(0), tempCount(0) {}

    bool generate(string& error) {
        try {
            string result = parseExpression();
            skipSpaces();

            if (pos != input.size()) {
                error = "Unexpected character '" +
                        string(1, input[pos]) +
                        "' at position " +
                        to_string(pos + 1) + ".";
                return false;
            }

            code.push_back({"=", result, "", "result"});
            return true;

        } catch (const string& message) {
            error = message;
            return false;
        }
    }

    const vector<Instruction>& getCode() const {
        return code;
    }
};

void display(const vector<Instruction>& code) {
    cout << "\nTHREE ADDRESS CODE\n";

    for (const auto& item : code) {
        if (item.op == "=") {
            cout << item.result << " = "
                 << item.arg1 << "\n";
        } else {
```

```
            cout << item.result << " = "
                 << item.arg1 << " "
                 << item.op << " "
                 << item.arg2 << "\n";
        }
    }

    cout << "\nQUADRUPLES\n";

    cout << left << setw(10) << "Operator"
         << setw(12) << "Arg1"
         << setw(12) << "Arg2"
         << setw(12) << "Result" << "\n";

    for (const auto& item : code) {
        cout << left << setw(10) << item.op
             << setw(12) << item.arg1
             << setw(12)
             << (item.arg2.empty() ? "-" : item.arg2)
             << setw(12) << item.result << "\n";
    }

    cout << "\nTRIPLES\n";

    cout << left << setw(8) << "Index"
         << setw(12) << "Operator"
         << setw(12) << "Arg1"
         << setw(12) << "Arg2" << "\n";

    for (size_t i = 0; i < code.size(); i++) {
        string a1 = code[i].arg1;
        string a2 = code[i].arg2.empty()
                        ? "-" : code[i].arg2;

        for (size_t j = 0; j < i; j++) {
            if (code[j].result == a1)
                a1 = "(" + to_string(j) + ")";

            if (code[j].result == a2)
                a2 = "(" + to_string(j) + ")";
        }

        cout << left << setw(8) << i
             << setw(12) << code[i].op
             << setw(12) << a1
             << setw(12) << a2 << "\n";
    }

    cout << "\nINDIRECT TRIPLES (Pointer Order)\n";
    cout << "Pointer\tTriple Index\n";

    for (size_t i = 0; i < code.size(); i++) {
        cout << "p" << i << "\t" << i << "\n";
    }
}

int main() {
    string expression;

    cout << "THREE ADDRESS CODE GENERATOR\n";
```

```
    cout << "Enter an arithmetic expression: ";

    getline(cin, expression);

    if (expression.empty()) {
        cout << "\nError: Expression cannot be empty.\n";
        return 1;
    }

    TACGenerator generator(expression);
    string error;

    if (!generator.generate(error)) {
        cout << "\nSyntax Error: " << error << "\n";
        return 1;
    }

    display(generator.getCode());

    return 0;
}
```

2. Results

The source code was compiled with a C++17 compiler and tested with two valid expressions and one invalid expression.

Test case	Input	Result
1	a + b * c	t1 = b * c; t2 = a + t1; result = t2
2	(a + b) * (c - d)	t1 = a + b; t2 = c - d; t3 = t1 * t2; result = t3
3	a + * b	Syntax error reported for invalid expression.

Example 1: a + b * c

```
THREE ADDRESS CODE GENERATOR
Enter an arithmetic
expression: THREE ADDRESS CODE
t1 = b * c t2 = a + t1 result
= t2

QUADRUPLES
Operator  Arg1        Arg2        Result
*         b           c           t1
+         a           t1          t2
=         t2          -           result

TRIPLES
Index   Operator    Arg1        Arg2
0       *           b           c
1       +           a           (0)
2       =           (1)         -

INDIRECT TRIPLES (Pointer
Order)


Pointer    Triple Index

 p0    0

 p1    1

 p2    2
```

Example 2: (a + b) * (c - d)

```
THREE ADDRESS CODE GENERATOR
Enter an arithmetic
expression: THREE ADDRESS CODE
t1 = a + b

 t2 = c - d

t3 = t1 * t2

result = t3

QUADRUPLES
Operator  Arg1        Arg2        Result
+         a           b           t1
-         c           d           t2
*         t1          t2          t3
=         t3          -           result

TRIPLES
Index   Operator    Arg1        Arg2
0       +           a           b
1       -           c           d
2       *           (0)         (1)
3       =           (2)         -

INDIRECT TRIPLES (Pointer
Order)


Pointer    Triple Index


 p0    0


p1    1


p2    2


p3    3
```

Example 3: Invalid expression a + * b

```
THREE ADDRESS CODE GENERATOR
Enter an arithmetic expression:
Syntax Error: Invalid arithmetic expression.
```

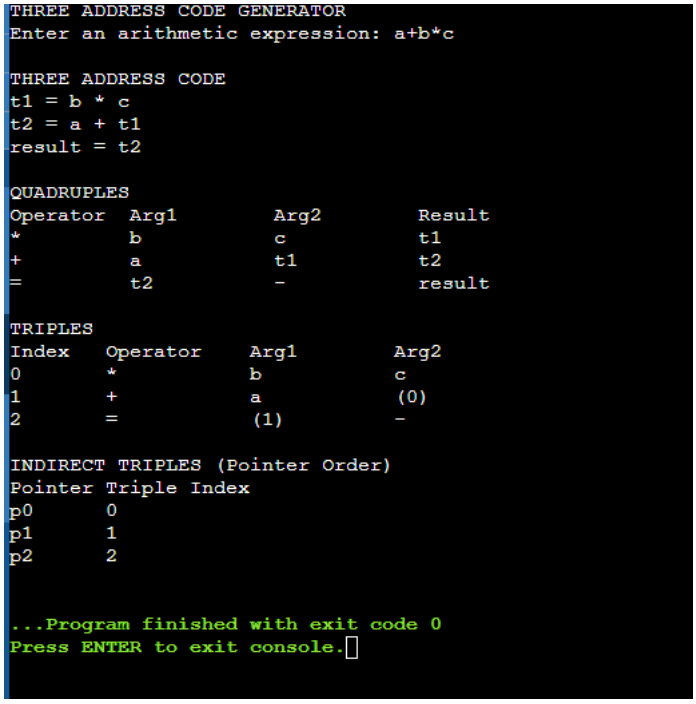
Conclusion

The program demonstrates expression parsing and intermediate-code generation. It generates TAC, Quadruples, Triples, and an Indirect Triple pointer sequence, while identifying basic malformed expressions.

3. Demo Screenshots

These screenshots show the captured console output produced by the compiled program for the three test cases.

Demo 1 - a + b * c



```
THREE ADDRESS CODE GENERATOR
Enter an arithmetic expression: a+b*c

THREE ADDRESS CODE
t1 = b * c
t2 = a + t1
result = t2

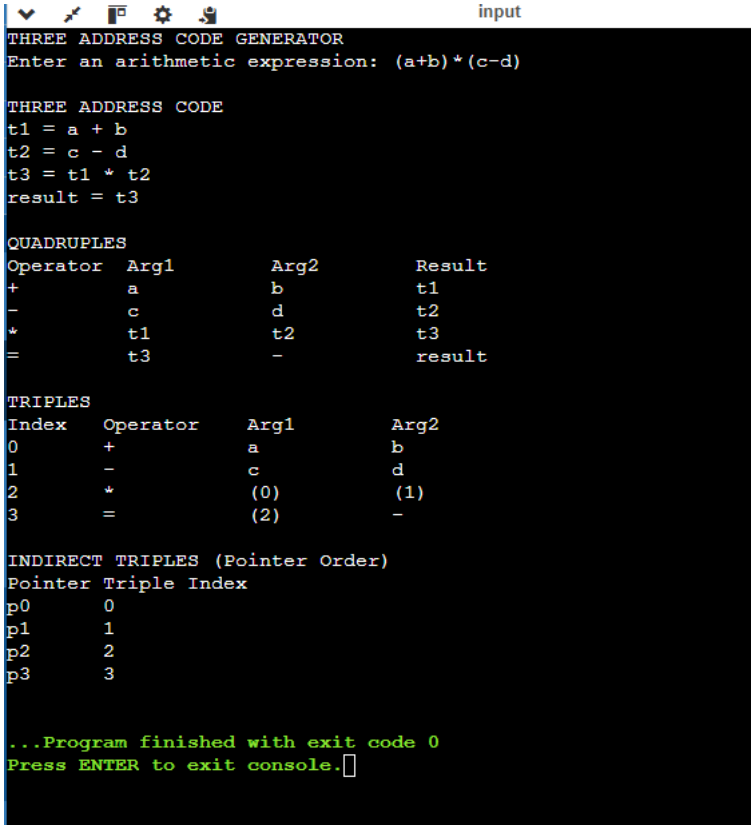
QUADRUPLES
Operator  Arg1        Arg2        Result
*         b           c           t1
+         a           t1          t2
=         t2          -           result

TRIPLES
Index     Operator    Arg1        Arg2
0         *           b           c
1         +           a           (0)
2         =           (1)         -

INDIRECT TRIPLES (Pointer Order)
Pointer Triple Index
p0      0
p1      1
p2      2


...Program finished with exit code 0
Press ENTER to exit console.
```

Demo 2 - (a + b) * (c - d)



```
input
THREE ADDRESS CODE GENERATOR
Enter an arithmetic expression: (a+b)*(c-d)

THREE ADDRESS CODE
t1 = a + b
t2 = c - d
t3 = t1 * t2
result = t3

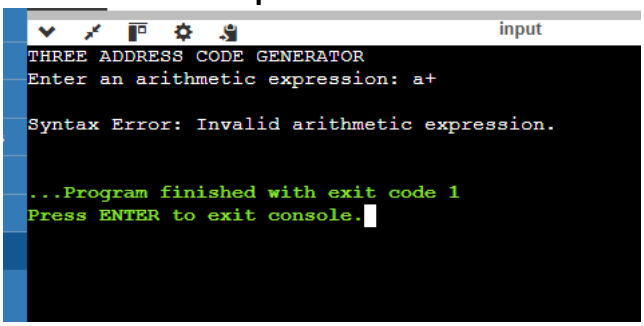
QUADRUPLES
Operator  Arg1        Arg2        Result
+         a           b           t1
-         c           d           t2
*         t1          t2          t3
=         t3          -           result

TRIPLES
Index     Operator    Arg1        Arg2
0         +           a           b
1         -           c           d
2         *           (0)         (1)
3         =           (2)         -

INDIRECT TRIPLES (Pointer Order)
Pointer Triple Index
p0      0
p1      1
p2      2
p3      3


...Program finished with exit code 0
Press ENTER to exit console.
```

Demo 3 - Invalid expression



How to Compile and Run

1. Save the code as **tac_generator.cpp**.
2. Open Command Prompt or a terminal in the folder containing the file.
3. Compile using: `g++ -std=c++17 tac_generator.cpp -o tac_generator`
4. Run the generated executable (on Windows, run tac_generator.exe; on Linux/macOS, use ./tac_generator).
5. Enter an arithmetic expression when prompted, for example `a + b * c`.

Submission checklist

Source code included
Results for multiple test cases included
Three demo screenshots included
Conclusion and run instructions included